

Objetos literales

MANUAL DE REFERENCIA · JAVASCRIPT · NIVEL BÁSICO

Agrupa datos y acciones en un solo objeto.

Creación, lectura, modificación, métodos con this, objetos anidados, copias, JSON y errores habituales, con ejemplos ejecutados de verdad.

01 Lee la explicación

02 Prueba el ejemplo en la consola

03 Cambia un valor y observa



JAVASCRIPT · NAVEGADOR · CONSOLA (F12)

Cada ejemplo muestra el código y, justo debajo, lo que escribe la consola al ejecutarlo. Complementa a las Prácticas Guiadas de objetos literales.

Alpe formación



Tu ruta de trabajo

ÍNDICE Y REQUISITOS

Este manual explica cómo crear y usar **objetos con notación literal** en JavaScript. Cada tema incluye código real y su salida en la consola. Escribe los ejemplos tú mismo y cambia valores: es la forma más rápida de aprender.



Temas

01 · Qué es un objeto literal 3–4

02 · Crear objetos 5–7

03 · Leer propiedades 8

04 · Modificar, añadir y borrar 9

05 · Métodos y this 10–11

06 · Objetos anidados 12

07 · Arrays de objetos 13

08 · Recorrer un objeto 14

09 · Referencias y copias 15

10 · Desestructuración y spread 16

11 · Getters, setters y privacidad 17–18

12 · Objetos literales y JSON 19–22

13 · Errores habituales 23

Chuleta de sintaxis: pág. 24 · Ejercicios: pág. 25 · Glosario: pág. 26 · Fuentes: pág. 27



Antes de empezar

- [ ] Conoces las variables (`let`, `const`), las funciones y los arrays de JavaScript.
- [ ] Sabes abrir la **consola del navegador** con **F12** y la pestaña **Consola**.
- [ ] Puedes probar los ejemplos en una página HTML con una etiqueta `<script>`, o directamente en la consola.
- [ ] Usas un editor de código, por ejemplo **Visual Studio Code**.



01 · Qué es un objeto literal

TEMA 01 · AGRUPAR DATOS Y ACCIONES

Un **objeto literal** agrupa bajo un mismo nombre las características (**propiedades**) y las acciones (**métodos**) de una misma entidad: un alumno, un producto, una cuenta bancaria.

Se llama **literal** porque lo escribes directamente en el código, entre llaves `{ ... }`, con pares `clave: valor` separados por comas. No necesitas una clase ni una función constructora. [1]



El problema: datos sueltos

VARIABLES SUELTAS Y ARRAYS

```
let nombre = "Ana";
let edad = 22;
let notas = [8, 9];

// Tres datos de la misma
// persona, sin relación entre sí
```

OBJETO LITERAL

```
const alumno = {
    nombre: "Ana",
    edad: 22,
    notas: [8, 9]
};

// Todo agrupado bajo
// un único nombre
```

Criterio	Variables o arrays sueltos	Objeto literal
Organización	Datos dispersos que pueden desincronizarse	Todo viaja unido bajo una única entidad
Comportamiento	Funciones sueltas que reciben todos los datos como parámetros	Los métodos viven en el objeto y usan sus datos con `this`
Listas	Varios arrays paralelos con el mismo índice	Un único array de objetos

Ventajas de los objetos

- ✔ Datos agrupados y ordenados.
- ✔ Acceso claro por nombre: `alumno.nombre` .
- ✔ Incluyen acciones, mediante métodos.

Cuándo no usarlos

- ✘ Se accede por clave, no por posición como en un array.
- ✘ Resultan más complejos para listas simples de valores.
- ✘ Para recorrerlos hay que obtener sus claves o sus entradas.



01 · Anatomía y vocabulario

TEMA 01 · PROPIEDADES, MÉTODOS Y NOTACIONES



Anatomía de un objeto literal

```
const coche = {
    marca: "Toyota",
    puertas: 5,
    arrancar() { return "En marcha"; }
};
```

Declaración const crea la variable · **Clave** nombre del miembro · **Valor** dato o función

Elemento	Qué es	Ejemplo
Llaves `{ }`	Delimitan el inicio y el final del objeto	`const coche = { ... };`
Miembro	Cualquier elemento interno: propiedad o método	`marca` y `arrancar()`
Propiedad	Miembro que guarda un dato (texto, número, booleano, array u otro objeto)	`marca: "Toyota"`
Método	Miembro que guarda una función: una acción del objeto	`arrancar() { ... }`
Notación de punto	La forma habitual de leer, modificar o añadir propiedades	`coche.marca`
Notación de corchetes	Permite usar cadenas o nombres guardados en variables	`coche["marca"]`

Fíjate: también se pueden crear objetos con `new Object()` o con clases, pero la notación literal es la más usada para agrupar datos sencillos. [1]



02 · Crear objetos

TEMA 02 · TRES FORMAS DE CREARLOS

Hay tres formas habituales de crear un objeto con notación literal. Las tres producen el mismo tipo de objeto.

crear-objetos.js

```js
// Forma 1: creación directa con sus propiedades (la más habitual)
const alumno = {
    nombre: "Elena",
    edad: 20,
    matriculado: true
};

// Forma 2: objeto vacío y asignación posterior
const ficha = {};
ficha.curso = "JavaScript Básico";
ficha.aula = 104;

// Forma 3: abreviada, a partir de variables existentes (ES6)
const marca = "Logitech";
const modelo = "MX Master";
const raton = { marca, modelo };   // equivale a { marca: marca, modelo: modelo }

console.log(alumno);
console.log(ficha);
console.log(raton);
```

SALIDA EN LA CONSOLA

```
› { nombre: 'Elena', edad: 20, matriculado: true }
  { curso: 'JavaScript Básico', aula: 104 }
  { marca: 'Logitech', modelo: 'MX Master' }
```

Forma	Cuándo usarla
1 · Directa	Conoces los datos desde el principio. Es la más habitual.
2 · Vacío y asignación	Los datos llegan después: por ejemplo, desde un formulario.
3 · Abreviada (ES6)	Ya tienes variables con el nombre que quieres como clave.



02 · Valores

TEMA 02 · QUÉ PUEDE GUARDAR UNA PROPIEDAD

El valor de una propiedad puede ser de **cualquier tipo**: texto, número, booleano, `null`, un array, otro objeto o una función (que se llama método).

tipos-de-valores.js

```
const demo = {
    texto: "Hola",
    numero: 42,
    booleano: true,
    nada: null,
    lista: [1, 2, 3],
    anidado: { clave: "valor" },
    accion() { return "Soy un método"; }
};

console.log(typeof demo.texto);
console.log(typeof demo.lista);
console.log(typeof demo.accion);
console.log(demo.accion());
```

SALIDA EN LA CONSOLA

```
› string
  object
  function
  Soy un método
```

Tipo de valor	Ejemplo
Texto, número, booleano	`"Hola"` · `42` · `true`
Ausencia de valor	`null`
Array	`[1, 2, 3]`
Otro objeto	`{ clave: "valor" }`
Función (método)	`accion() { ... }`



02 · Nombres de clave

TEMA 02 · CUÁNDO LLEVAN COMILLAS

Si la clave es un identificador válido, no necesita comillas. Si contiene espacios o guiones, va entre comillas y se lee con corchetes. Una clave entre corchetes se **calcula** a partir de una variable.

nombres-de-clave.js

```
const especial = {
    "nombre completo": "Ana Ruiz",    // espacios: necesita comillas
    "e-mail": "ana@ejemplo.com",      // guion: necesita comillas
    2024: "año",                      // un número también vale
    clase: "A"                        // identificador válido: sin comillas
};
console.log(especial["nombre completo"]);
console.log(especial["e-mail"]);
console.log(especial[2024]);
console.log(especial.clase);

const campo = "color";
const camiseta = { talla: "M", [campo]: "azul" };   // clave calculada
console.log(camiseta);
```

SALIDA EN LA CONSOLA

```
Ana Ruiz
ana@ejemplo.com
año
A
{ talla: 'M', color: 'azul' }
```

Clave	Cómo se escribe	Cómo se lee
Identificador válido	`clase: "A"`	`obj.clase`
Con espacios o guiones	`"e-mail": "..."`	`obj["e-mail"]`
Numérica	`2024: "año"`	`obj[2024]`
Calculada	`[campo]: "azul"`	`obj.color`



03 · Leer propiedades

TEMA 03 · PUNTO, CORCHETES Y ?.

Para leer una propiedad usas la **notación de punto** o la de **corchetes**. Los corchetes sirven cuando la clave está en una variable o tiene caracteres especiales.

leer-propiedades.js

```
const coche = {
    marca: "Toyota",
    modelo: "Yaris",
    puertas: 5,
    extras: { aire: true }
};

console.log(coche.marca);          // notación de punto
console.log(coche["modelo"]);      // notación de corchetes
const clave = "puertas";
console.log(coche[clave]);         // clave guardada en una variable

console.log(coche.color);          // no existe: undefined
console.log("marca" in coche);     // ¿existe la clave?
console.log("color" in coche);
console.log(coche.garantia?.anios);   // ?. evita el error si falta garantia
```

SALIDA EN LA CONSOLA

```
Toyota
Yaris
5
undefined
true
false
undefined
```

Necesitas…	Usa	Resultado si no existe
Leer una clave conocida	`obj.clave`	`undefined`
Leer una clave guardada en una variable	`obj[variable]`	`undefined`
Saber si una clave existe	`"clave" in obj`	`false`
Leer un dato anidado que quizá falte	`obj.a?.b`	`undefined`, sin error

Fíjate: `obj.propiedadVariable` busca una clave llamada literalmente «propiedadVariable». Para usar el valor de una variable, escribe `obj[propiedadVariable]`.



04 · Modificar, añadir y borrar

TEMA 04 · OBJETOS DINÁMICOS

Un objeto es **dinámico**: puedes cambiar el valor de una propiedad, añadir otras nuevas o borrarlas cuando quieras.

modificar-objetos.js

```
1  const producto = {
2      codigo: "A-104",
3      nombre: "Teclado Mecánico",
4      precio: 59.90,
5      enStock: true
6  };
7
8  producto.precio = 49.95;            // modificar
9  producto.categoria = "Periféricos"; // añadir
10 delete producto.codigo;             // borrar
11 console.log(producto);
12
13 // const impide reasignar la variable, no modificar su contenido
14 try {
15     producto = {};
16 } catch (e) {
17     console.log(e.name + ": " + e.message);
18 }
19
20 const fijo = Object.freeze({ iva: 21 });   // objeto congelado
21 fijo.iva = 10;                              // se ignora
22 console.log(fijo.iva);
```

SALIDA EN LA CONSOLA

```
› {
    nombre: 'Teclado Mecánico',
    precio: 49.95,
    enStock: true,
    categoria: 'Periféricos'
  }
  TypeError: Assignment to constant variable.
  21
```

Fíjate: `const` impide reasignar la variable (`producto = {}` da error), pero **sí permite** modificar, añadir o borrar propiedades. Para impedirlo, usa `Object.freeze()` . En modo estricto, modificar un objeto congelado lanza un error.



05 · Métodos y this

TEMA 05 · FUNCIONES DENTRO DE OBJETOS

Un **método** es una función guardada en una propiedad. Cuando lo llamas con `objeto.metodo()`, la palabra `this` apunta a **ese objeto**. [2]

metodos-y-this.js

```
1  const cuenta = {
2      titular: "Lucía Navarro",
3      saldo: 200,
4      ingresar(cantidad) {
5          this.saldo += cantidad;
6          return `Ingreso de ${cantidad}€. Saldo actual: ${this.saldo}€`;
7      },
8      retirar(cantidad) {
9          if (cantidad > this.saldo) {
10             return "Operación denegada: saldo insuficiente.";
11         }
12         this.saldo -= cantidad;
13         return `Retirada de ${cantidad}€. Saldo restante: ${this.saldo}€`;
14     }
15 };
16
17 console.log(cuenta.ingresar(50));
18 console.log(cuenta.retirar(100));
19 console.log(cuenta.retirar(500));
```

SALIDA EN LA CONSOLA

```
Ingreso de 50€. Saldo actual: 250€
Retirada de 100€. Saldo restante: 150€
Operación denegada: saldo insuficiente.
```

Sintaxis	Ejemplo
Método abreviado (la recomendada)	`saludar() { return "Hola"; }`
Función como valor	`saludar: function () { return "Hola"; }`
Función flecha (**no** para métodos que usan `this`)	`saludar: () => "Hola"`



05 · Errores con this

TEMA 05 · FLECHAS Y MÉTODOS SEPARADOS

El valor de `this` depende de **cómo se llama** a la función, no de dónde está escrita. Dos errores clásicos:

errores-con-this.js

```
1  const gato = {
2      nombre: "Misi",
3      saludarMal: () => `Soy ${this.nombre}`,        // flecha: this NO es el objeto
4      saludarBien() { return `Soy ${this.nombre}`; }  // método abreviado: correcto
5  };
6  console.log(gato.saludarBien());
7  console.log(gato.saludarMal());
8
9  // Perder this al separar el método del objeto
10 const suelto = gato.saludarBien;
11 console.log(suelto());
12
13 // Solución: fijar this con bind
14 const ligado = gato.saludarBien.bind(gato);
15 console.log(ligado());
```

SALIDA EN LA CONSOLA

```
› Soy Misi
  Soy undefined
  Soy undefined
  Soy Misi
```

Error	Por qué ocurre	Solución
Método escrito con **función flecha**	Las funciones flecha no tienen su propio `this`: usan el del contexto exterior. [3]	Usar el método abreviado `saludar() { ... }`
Perder `this` al separar el método	Al llamar `suelto()` ya no hay un objeto delante del punto.	Llamarlo siempre como `objeto.metodo()` o fijarlo con `.bind(objeto)`



06 · Objetos anidados

TEMA 06 · OBJETOS Y ARRAYS DENTRO DE OBJETOS

Un objeto puede contener **otros objetos** y **arrays**. Para llegar a un dato interno encadenas puntos y corchetes.

objetos-anidados.js

```
const estudiante = {
    nombre: "Carlos Ruiz",
    direccion: {
        calle: "Av. de los Castros, 42",
        ciudad: "Santander",
        cp: "39005"
    },
    calificaciones: [7.5, 8.0, 9.5],
    calcularMedia() {
        let suma = 0;
        for (const nota of this.calificaciones) {
            suma += nota;
        }
        return (suma / this.calificaciones.length).toFixed(2);
    }
};

console.log(estudiante.direccion.ciudad);
console.log(estudiante.calificaciones[0]);
console.log(estudiante.calcularMedia());

estudiante.direccion.ciudad = "Torrelavega";
console.log(estudiante.direccion);
```

SALIDA EN LA CONSOLA

```
Santander
7.5
8.33
{ calle: 'Av. de los Castros, 42', ciudad: 'Torrelavega', cp: '39005' }
```

Fíjate: si un nivel intermedio no existe, `obj.a.b` lanza un `TypeError`. Con `obj.a?.b` obtienes `undefined` sin error.



07 · Arrays de objetos

TEMA 07 · COLECCIONES DE ENTIDADES

La estructura más habitual en programación web es un **array de objetos** con las mismas claves: catálogos, listas de usuarios, tareas... Los métodos de array se combinan con ellos.

array-de-objetos.js

```
const catalogo = [
    { id: 1, titulo: "Monitor 24 pulgadas", precio: 140, disponible: true },
    { id: 2, titulo: "Ratón Inalámbrico", precio: 25, disponible: false },
    { id: 3, titulo: "Alfombrilla XXL", precio: 18, disponible: true }
];

for (const item of catalogo) {
    if (item.disponible) {
        console.log(`✔ ${item.titulo} - ${item.precio}€`);
    }
}

const baratos = catalogo.filter(p => p.precio < 50);
const titulos = catalogo.map(p => p.titulo);
const raton = catalogo.find(p => p.id === 2);
const total = catalogo.reduce((suma, p) => suma + p.precio, 0);

console.log(baratos.length);
console.log(titulos);
console.log(raton.titulo);
console.log(total);
```

SALIDA EN LA CONSOLA

```
✔ Monitor 24 pulgadas - 140€
✔ Alfombrilla XXL - 18€
2
[ 'Monitor 24 pulgadas', 'Ratón Inalámbrico', 'Alfombrilla XXL' ]
Ratón Inalámbrico
183
```

Método	Qué devuelve
`filter(fn)`	Un array nuevo con los objetos que cumplen la condición
`map(fn)`	Un array nuevo con el resultado de transformar cada objeto
`find(fn)`	El primer objeto que cumple la condición (o `undefined`)
`reduce(fn, inicial)`	Un único valor acumulado: una suma, un total...



08 · Recorrer un objeto

TEMA 08 · KEYS, VALUES, ENTRIES Y FOR...IN

Un objeto no es iterable directamente. Para recorrerlo, obtienes sus claves, sus valores o sus entradas (parejas `[clave, valor]`).

recorrer-un-objeto.js

```js
const configuracion = {
    tema: "oscuro",
    idioma: "es",
    notificaciones: true
};

console.log(Object.keys(configuracion));
console.log(Object.values(configuracion));
console.log(Object.entries(configuracion));

for (const clave in configuracion) {
    console.log(clave + " → " + configuracion[clave]);
}

for (const [clave, valor] of Object.entries(configuracion)) {
    console.log(`${clave}: ${valor}`);
}
```

SALIDA EN LA CONSOLA

```
[ 'tema', 'idioma', 'notificaciones' ]
[ 'oscuro', 'es', true ]
[
  [ 'tema', 'oscuro' ],
  [ 'idioma', 'es' ],
  [ 'notificaciones', true ]
]
tema → oscuro
idioma → es
notificaciones → true
tema: oscuro
idioma: es
notificaciones: true
```

Herramienta	Qué te da
`Object.keys(obj)`	Array con las claves
`Object.values(obj)`	Array con los valores
`Object.entries(obj)`	Array de parejas `[clave, valor]`
`for (const k in obj)`	Recorre las claves una a una



09 · Referencias y copias

TEMA 09 · ASIGNAR NO ES COPIAR

Una variable no guarda el objeto: guarda una **referencia** a él. Por eso `const b = a` **no copia** nada: `a` y `b` apuntan al mismo objeto. Copiar un valor en JavaScript casi siempre es superficial. [4]

copiar-objetos.js

```
1  const a = { nombre: "Ana", notas: [7, 8] };
2
3  const b = a;                    // NO es una copia: apunta al mismo objeto
4  b.nombre = "Berta";
5  console.log(a.nombre);
6  console.log(a === b);
7
8  const c = { ...a };             // copia superficial
9  c.nombre = "Carla";
10 c.notas.push(10);               // el array interno se comparte
11 console.log(a.nombre, a.notas);
12
13 const d = structuredClone(a);   // copia profunda
14 d.notas.push(5);
15 console.log(a.notas, d.notas);
16
17 console.log({} === {});         // dos objetos distintos nunca son iguales
```

SALIDA EN LA CONSOLA

```
› Berta
  true
  Berta [ 7, 8, 10 ]
  [ 7, 8, 10 ] [ 7, 8, 10, 5 ]
  false
```

Técnica	Tipo de copia	Cuidado con…
`const b = a`	Ninguna: misma referencia	Cualquier cambio afecta a los dos
`{ ...a }`	Superficial	Los objetos y arrays internos se comparten
`structuredClone(a)`	Profunda	Lanza un error si el objeto contiene funciones [4]



10 · Desestructuración y spread

TEMA 10 · EXTRAER Y COMBINAR PROPIEDADES

La **desestructuración** extrae propiedades en variables sueltas. El operador **spread** (`...`) copia las propiedades de un objeto dentro de otro.

desestructuracion-y-spread.js

```
1  const alumno = { nombre: "Elena", edad: 20, curso: "DAW" };
2
3  const { nombre, edad } = alumno;                  // extraer propiedades
4  const { curso: titulacion, aula = 104 } = alumno;  // renombrar y valor por defecto
5  console.log(nombre, edad, titulacion, aula);
6
7  const base = { tema: "claro", idioma: "es" };
8  const mia = { ...base, tema: "oscuro", notificaciones: true };  // combinar
9  console.log(mia);
10
11 function presentar({ nombre, edad = 18 }) {        // objeto como argumento
12     return `${nombre} tiene ${edad} años`;
13 }
14 console.log(presentar(alumno));
15 console.log(presentar({ nombre: "Marcos" }));
```

SALIDA EN LA CONSOLA

```
› Elena 20 DAW 104
  { tema: 'oscuro', idioma: 'es', notificaciones: true }
  Elena tiene 20 años
  Marcos tiene 18 años
```

Sintaxis	Qué hace
`const { a, b } = obj`	Crea las variables `a` y `b` con los valores de `obj.a` y `obj.b`
`const { a: nuevo } = obj`	Extrae `a` y la guarda con otro nombre
`const { a = 1 } = obj`	Da un valor por defecto si `a` no existe
`{ ...base, tema: "oscuro" }`	Copia `base` y sobrescribe o añade propiedades
`function f({ a, b })`	Recibe un objeto como argumento y extrae sus propiedades



11 · Getters y setters

TEMA 11 · PROPIEDADES CALCULADAS Y PRIVACIDAD

Un **getter** (`get`) y un **setter** (`set`) parecen propiedades normales, pero ejecutan una función al leerlas o asignarlas. Sirven para calcular valores o validar datos.

getters-y-setters.js

```
const circulo = {
    radio: 5,
    get diametro() {
        return this.radio * 2;
    },
    set diametro(valor) {
        this.radio = valor / 2;
    }
};

console.log(circulo.diametro);   // se lee como una propiedad
circulo.diametro = 40;           // se asigna como una propiedad
console.log(circulo.radio);
```

SALIDA EN LA CONSOLA

```
10
20
```



Privacidad en un objeto literal

Todos los miembros de un objeto literal son **públicos**. La sintaxis privada `#campo` solo funciona dentro de una `class` : en un objeto literal provoca un error de sintaxis. Para proteger datos hay dos opciones:

- **Convención `_` :** un guion bajo delante avisa de que no debe tocarse, pero no lo impide.
- **Clausura:** una función autoejecutable devuelve un objeto que solo expone lo público. Las variables internas quedan inaccesibles.



11 · Privacidad real con clausuras

TEMA 11 · CONVENCIÓN FRENTE A CLAUSURA

privacidad.js

```
// A) Convención: el guion bajo avisa, pero no protege
const usuario = {
    nombre: "Marta",
    _claveSecreta: "1234",
    saludar() { return `Hola, soy ${this.nombre}`; }
};
console.log(usuario._claveSecreta);

// B) Privacidad real con una clausura (función autoejecutable)
const cajaFuerte = (function () {
    const pinSecreto = "9876";
    let dinero = 1000;

    function registrar(texto) {
        console.log("[Auditoría] " + texto);
    }

    return {
        consultarSaldo(pin) {
            if (pin !== pinSecreto) return "PIN incorrecto";
            registrar("Consulta de saldo autorizada");
            return dinero + "€";
        }
    };
})();

console.log(cajaFuerte.dinero);
console.log(cajaFuerte.consultarSaldo("0000"));
console.log(cajaFuerte.consultarSaldo("9876"));
```

SALIDA EN LA CONSOLA

```
1234
undefined
PIN incorrecto
[Auditoría] Consulta de saldo autorizada
1000€
```

Fíjate: `cajaFuerte.dinero` da `undefined`: la variable existe dentro de la clausura, pero el objeto devuelto no la expone. Solo se llega al dato a través del método público `consultarSaldo()`.



12 · Objetos literales y JSON

TEMA 12 · DOS SINTAXIS PARECIDAS, DOS USOS

JSON es un formato de **texto plano** para intercambiar datos. Su sintaxis se parece a la de un objeto literal, pero es distinta: lo que es válido en JavaScript no siempre lo es en JSON. [5]

OBJETO LITERAL (en memoria)

```
const alumno = {
    nombre: "Ana",
    edad: 22,
    saludar() { return "Hola"; }
};   // admite comentarios
```

JSON (texto)

```
{
    "nombre": "Ana",
    "edad": 22
}
```

	Objeto literal de JS	JSON
Qué es	Una estructura viva en la memoria de JavaScript	Una cadena de texto plano
Uso	Código vivo en JavaScript	Intercambio y almacenamiento de datos
Claves	Sin comillas si son identificadores válidos	Siempre entre comillas dobles `"..."`
Métodos	Permitidos	Prohibidos
Comentarios	Permitidos	No admite
Textos	Comillas simples, dobles o invertidas	Solo comillas dobles



Convertir de uno a otro

Función	Convierte	Ejemplo
`JSON.stringify(obj)`	Objeto → texto JSON	`{"nombre":"Ana"}`
`JSON.parse(texto)`	Texto JSON → objeto	`{ nombre: "Ana" }`



12 · Convertir con JSON

TEMA 12 · STRINGIFY Y PARSE

`JSON.stringify` convierte un objeto en **texto**. Con el segundo y tercer argumento (`null, 2`) el texto sale con sangría de dos espacios, más fácil de leer. `JSON.parse` hace el camino contrario.

objeto-a-json.js

```
1  const alumno = { nombre: "Ana", edad: 22, notas: [8, 9], activo: true };
2
3  const texto = JSON.stringify(alumno);   // objeto → texto JSON
4  console.log(texto);
5  console.log(typeof texto);
6
7  const copia = JSON.parse(texto);        // texto JSON → objeto
8  console.log(copia.nombre, copia.notas[1]);
9
10 console.log(JSON.stringify(alumno, null, 2));   // con sangría para leerlo mejor
```

SALIDA EN LA CONSOLA

```
› {"nombre":"Ana","edad":22,"notas":[8,9],"activo":true}
  string
  Ana 9
  {
    "nombre": "Ana",
    "edad": 22,
    "notas": [
      8,
      9
    ],
    "activo": true
  }
```

Llamada	Resultado
`JSON.stringify(obj)`	Texto en una sola línea, el más compacto
`JSON.stringify(obj, null, 2)`	Texto con sangría de 2 espacios
`typeof texto`	`"string"`: el JSON es solo texto



12 · Lo que JSON pierde

TEMA 12 · FUNCIONES, FECHAS Y ERRORES

No todo sobrevive al viaje de ida y vuelta. Fíjate en lo que desaparece o cambia de tipo.

lo-que-json-pierde.js

```
1  const objeto = {
2      nombre: "Ana",
3      saludar() { return "Hola"; },
4      pendiente: undefined,
5      registro: new Date("2026-01-15T10:30:00Z")
6  };
7
8  const texto = JSON.stringify(objeto);
9  console.log(texto);
10
11 const vuelta = JSON.parse(texto);
12 console.log(typeof vuelta.registro);
13 console.log(vuelta.saludar);
14
15 try {
16     JSON.parse("{nombre: 'Ana'}");   // JSON inválido: sin comillas dobles
17 } catch (e) {
18     console.log(e.name);
19 }
```

SALIDA EN LA CONSOLA

```
› {"nombre":"Ana","registro":"2026-01-15T10:30:00.000Z"}
  string
  undefined
  SyntaxError
```

Fíjate: `JSON.stringify` descarta las funciones y los valores `undefined`, y convierte las fechas en texto. `JSON.parse` lanza un `SyntaxError` si el texto no es JSON válido. [6]



12 · JSON y localStorage

TEMA 12 · GUARDAR OBJETOS EN EL NAVEGADOR

`localStorage` guarda **solo texto**, así que para guardar un objeto lo conviertes antes a JSON y lo reconstruyes al leerlo. [7] Este código funciona en el navegador, no en Node.

guardar-en-localstorage.js

```
1  const alumno = { nombre: "Ana", edad: 22 };
2
3  // Guardar: localStorage solo admite texto
4  localStorage.setItem("alumno", JSON.stringify(alumno));
5
6  // Recuperar: texto → objeto
7  const recuperado = JSON.parse(localStorage.getItem("alumno"));
8  console.log(recuperado.nombre);
```

Paso	Instrucción
Guardar	`localStorage.setItem("clave", JSON.stringify(objeto))`
Leer	`JSON.parse(localStorage.getItem("clave"))`
Si aún no hay datos	`getItem` devuelve `null`: usa `|| []` o `|| {}` como valor inicial



13 · Errores habituales

TEMA 13 · ANTES DE PEDIR AYUDA

Estos son los fallos más frecuentes al empezar. Los mensajes son los de Chrome y Edge; en Firefox cambia el texto, no la causa.

Error	Causa	Forma correcta
`{ nombre = "Ana" }`	Dentro de las llaves se usan dos puntos `:`, no el signo igual	`{ nombre: "Ana" }`
Falta una coma entre propiedades	`SyntaxError: Unexpected identifier 'b'`	Separar todas las propiedades con comas
`return saldo;` en un método	`ReferenceError: saldo is not defined`: falta `this.`	`return this.saldo;`
Método con función flecha	La flecha no vincula `this` al objeto	Método abreviado `saludar() { ... }`
`obj.variable` con una variable	El punto busca una clave llamada literalmente así	`obj[variable]`
`obj.a.b` con `a` inexistente	`TypeError: Cannot read properties of undefined (reading 'b')`	`obj.a?.b`
`const b = a` esperando una copia	Ambas variables apuntan al mismo objeto	`{ ...a }` o `structuredClone(a)`
`{} === {}` esperando `true`	Dos objetos distintos nunca son iguales	Comparar propiedades concretas
Claves repetidas	La última sobrescribe a las anteriores	Revisar los nombres de clave
`JSON.parse("{'x':1}")`	`SyntaxError`: JSON exige comillas dobles	`{"x":1}`



Chuleta de sintaxis

REFERENCIA RÁPIDA · OBJETOS LITERALES

Resumen de la sintaxis que has visto en este manual.

Propiedad	Para qué sirve	Ejemplo
`{ clave: valor }`	Crea un objeto literal	`const a = { nombre: "Ana" };`
`{ x, y }`	Abreviatura con variables existentes	`const p = { x, y };`
`{ [expr]: valor }`	Clave calculada	`{ [campo]: "azul" }`
`obj.clave`	Lee con punto	`a.nombre`
`obj["clave"]`	Lee con corchetes	`a[campo]`
`obj.clave = valor`	Modifica o añade	`a.edad = 22;`
`delete obj.clave`	Borra una propiedad	`delete a.edad;`
`"clave" in obj`	Comprueba si existe	`"edad" in a`
`obj?.a?.b`	Acceso seguro a datos anidados	`a.direccion?.ciudad`
`metodo() { }`	Método abreviado	`saludar() { return this.nombre; }`
`this`	El objeto que llama al método	`this.saldo += n;`
`get` / `set`	Propiedades calculadas	`get total() { ... }`
`Object.keys(obj)`	Array de claves	`Object.keys(a)`
`Object.values(obj)`	Array de valores	`Object.values(a)`
`Object.entries(obj)`	Array de parejas	`for (const [k, v] of Object.entries(a))`
`Object.freeze(obj)`	Impide modificar el objeto	`Object.freeze(config);`
`Object.assign(dest, src)`	Copia propiedades al destino	`Object.assign({}, a)`
`{ ...a }`	Copia superficial / combinar	`{ ...base, tema: "oscuro" }`
`structuredClone(obj)`	Copia profunda	`structuredClone(a)`
`const { a, b } = obj`	Desestructuración	`const { nombre } = a;`
`JSON.stringify(obj)`	Objeto → texto JSON	`JSON.stringify(a)`
`JSON.parse(texto)`	Texto JSON → objeto	`JSON.parse(texto)`



Practica lo aprendido

EJERCICIOS PROPUESTOS

Crea un archivo HTML con una etiqueta `<script>` y comprueba cada ejercicio en la consola del navegador (**F12**).



Ejercicios propuestos

- [ ] **01 · Libro.** Crea un objeto `libro` con `titulo`, `autor`, `paginas` y `leido`. Muéstralo con `console.log`.
- [ ] **02 · Claves variables.** Guarda el nombre de una propiedad en una variable y léela con corchetes.
- [ ] **03 · Cambios.** Modifica `leido`, añade `genero`, borra `paginas` y comprueba el resultado con `in`.
- [ ] **04 · Método.** Añade `resumen()`, que devuelva el título y el autor usando `this`.
- [ ] **05 · Anidados.** Crea un objeto `empresa` con una `direccion` y un array de `empleados` (objetos).
- [ ] **06 · Catálogo.** Con un array de productos, obtén los más baratos con `filter` y el total con `reduce`.
- [ ] **07 · Copias.** Demuestra la diferencia entre `b = a`, `{ ...a }` y `structuredClone(a)`.
- [ ] **08 · Opciones.** Escribe una función que reciba un objeto de opciones con valores por defecto.
- [ ] **09 · JSON.** Convierte un objeto a texto, recupéralo y guarda el resultado en `localStorage`.
- [ ] **10 · Contador privado.** Crea con una clausura un contador cuyo valor no se pueda modificar desde fuera.

Siguiente paso: aplica todo esto en las **Prácticas Guiadas**: una ficha de personaje, un carrito de compra y una agenda de contactos con JSON.



Glosario

TÉRMINOS CLAVE

Término	Definición
Objeto literal	Objeto creado escribiendo sus propiedades entre llaves `{ ... }`.
Propiedad	Miembro que guarda un dato asociado a una clave.
Método	Miembro que guarda una función: una acción del objeto.
Miembro	Cualquier elemento interno de un objeto: propiedad o método.
Clave	Nombre con el que se accede a un miembro.
`this`	Palabra clave que, dentro de un método, apunta al objeto que lo llama.
Notación de punto	Acceso con `obj.clave`.
Notación de corchetes	Acceso con `obj["clave"]` o `obj[variable]`.
Referencia	Dirección en memoria de un objeto: una variable guarda la referencia, no el objeto.
Copia superficial	Copia las propiedades de primer nivel; los objetos internos se comparten.
Copia profunda	Copia también los objetos internos de forma recursiva.
Desestructuración	Extraer propiedades de un objeto en variables: `const { a } = obj`.
Spread	Operador `...` que expande las propiedades de un objeto.
Clausura	Función que recuerda las variables de su entorno; permite datos privados.
JSON	Formato de texto para intercambiar datos, con claves y textos entre comillas dobles.
Serializar	Convertir un objeto en texto (`JSON.stringify`).



Fuentes y Documentación Oficial

REFERENCIAS DEL MANUAL · CONSULTA: 13/11/2026

Cada número entre corchetes remite a la documentación correspondiente. Los ejemplos de este manual se han ejecutado en Node.js y en el navegador.



Fuentes y Documentación Oficial

[1] Trabajando con objetos · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Guide/Trabajando_con_objectos

[2] this · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Operadores/this

[3] Funciones flecha · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Reference/Functions/Arrow_functions

[4] Copia profunda en JavaScript con structuredClone · web.dev · web.dev/articles/structured-clone

[5] JSON · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/JSON

[6] JSON.stringify() · MDN Web Docs · developer.mozilla.org/JavaScript/Reference/Global_Objects/JSON/stringify

[7] Window.localStorage · MDN Web Docs · developer.mozilla.org/es/docs/Web/API/Window/localStorage